

ECEN 454 - Lab Report

Lab Number: 1

Lab Title: Introduction to Cadence Schematic Design & Simulation

Section Number: 341

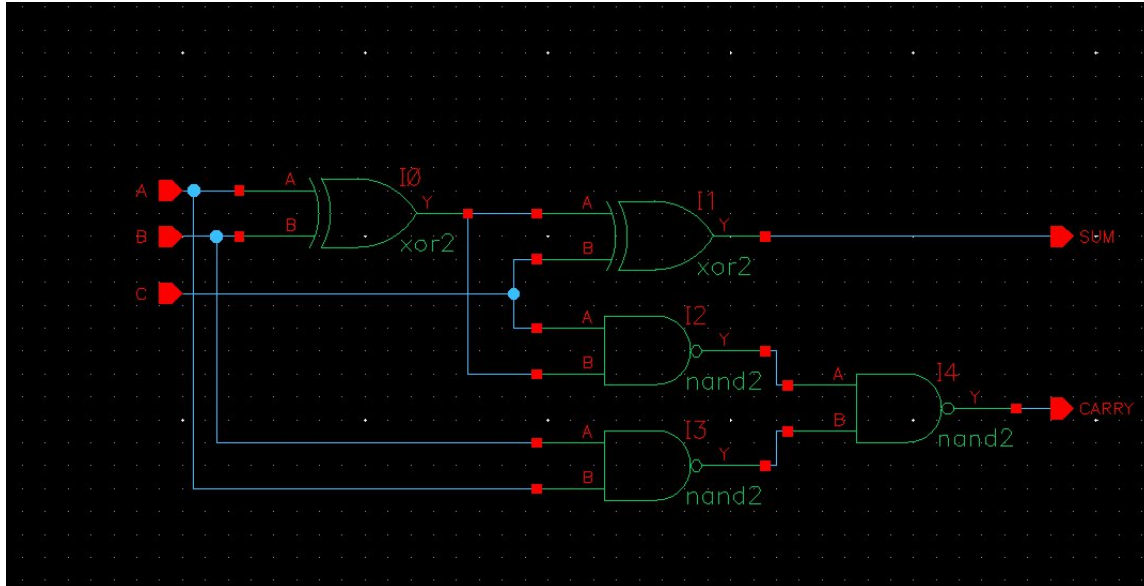
Student's Name: Abhishek Bhattacharyya

Student's UIN: 731002289

Date: 9/14/2024

TA: Nishank Bansal

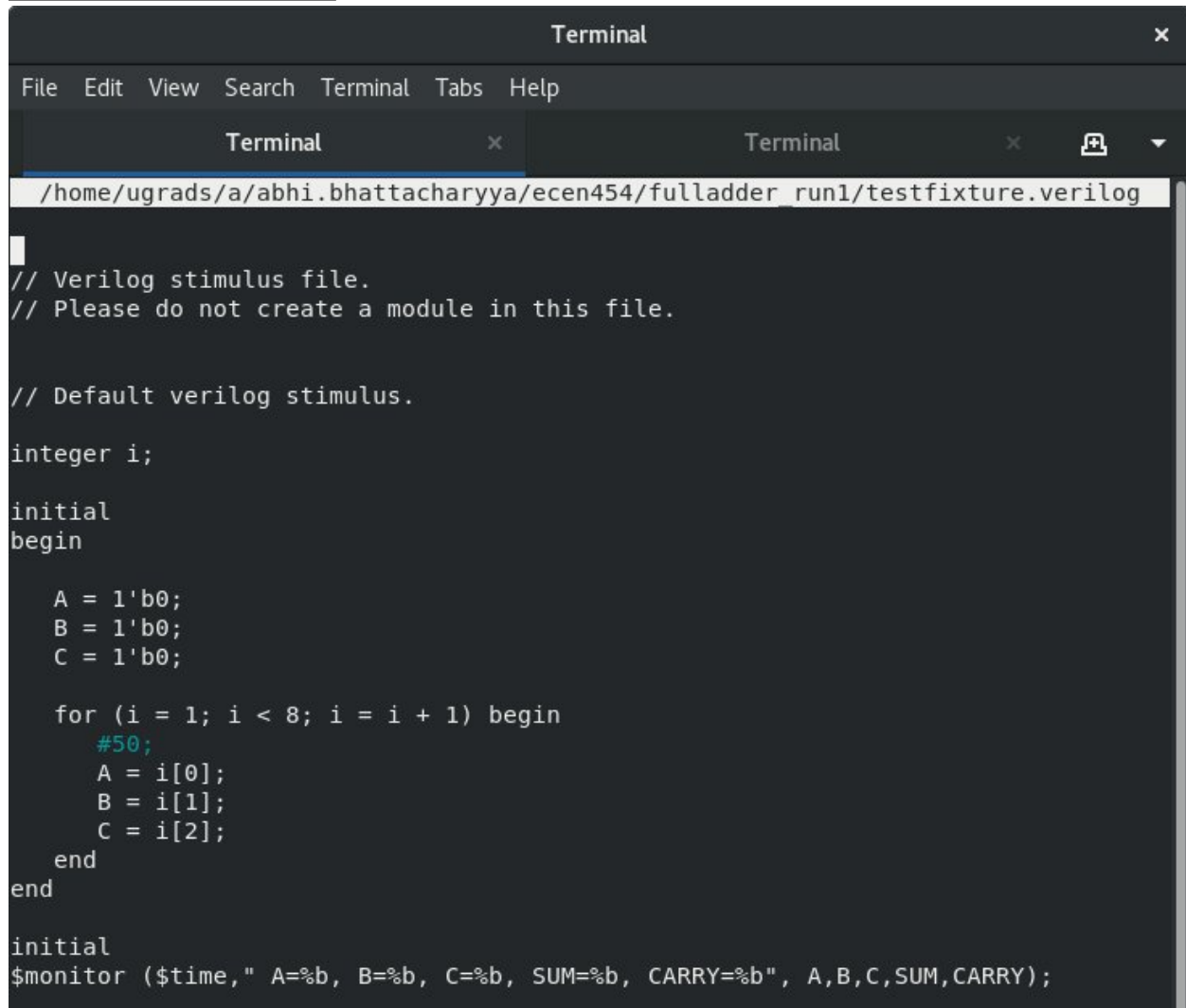
Full Adder Schematic



Full Adder Symbol



Full Adder Test Stimulus



The image shows a terminal window with a dark background and light text. The window title is "Terminal" with a close button. The menu bar includes "File", "Edit", "View", "Search", "Terminal", "Tabs", and "Help". There are two tabs open, both titled "Terminal", with a close button and a refresh icon. The active tab shows the file path `/home/ugrads/a/abhi.bhattacharyya/ecen454/fulladder_run1/testfixture.verilog`. The content of the file is a Verilog test stimulus. It starts with comments: `// Verilog stimulus file.` and `// Please do not create a module in this file.`. This is followed by `// Default verilog stimulus.`. The code then declares an integer `i`, initializes it, and enters a loop from `i = 1` to `i = 8`. Inside the loop, it assigns `A = i[0]`, `B = i[1]`, and `C = i[2]` after a `#50` delay. The loop ends with `end`. Finally, there is an `initial` block containing a `$monitor` statement that prints the values of `A`, `B`, `C`, `SUM`, and `CARRY` at each iteration.

```
Terminal
File Edit View Search Terminal Tabs Help
Terminal
/home/ugrads/a/abhi.bhattacharyya/ecen454/fulladder_run1/testfixture.verilog

// Verilog stimulus file.
// Please do not create a module in this file.

// Default verilog stimulus.

integer i;

initial
begin

    A = 1'b0;
    B = 1'b0;
    C = 1'b0;

    for (i = 1; i < 8; i = i + 1) begin
        #50;
        A = i[0];
        B = i[1];
        C = i[2];
    end
end

initial
$monitor ($time," A=%b, B=%b, C=%b, SUM=%b, CARRY=%b", A,B,C,SUM,CARRY);
```

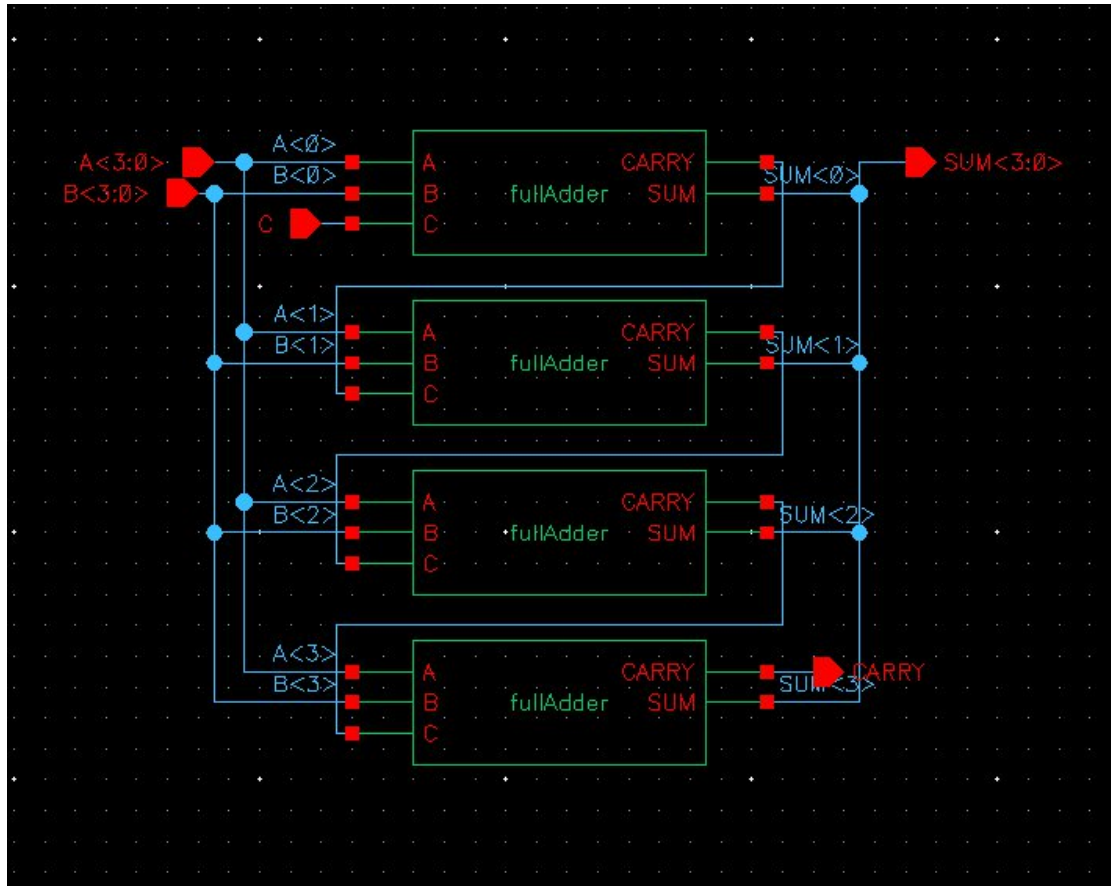
Full Adder Sim Results vs. Expected Results Truth Table

```
Terminal
File Edit View Search Terminal Tabs Help
Terminal x Terminal x [icon] v
bash-4.4$ cat simout.tmp
T00L: ncxlmode 15.20-s086: Started on Sep 12, 2024 at 15:07:30 CDT
ncxlmode
+delay_mode_path
+typdelays
-l
simout.tmp
/home/ugrads/a/abhi.bhattacharyya/ecen454/fulladder_run1/testfixture.tem
plate
-f /home/ugrads/a/abhi.bhattacharyya/ecen454/fulladder_run1/verilog.inpf
iles
/opt/coe/ncsu/ncsu-cdk-1.6.0.beta/lib/NCSU_Digital_Parts/nand2/f
unctional/verilog.v
/opt/coe/ncsu/ncsu-cdk-1.6.0.beta/lib/NCSU_Digital_Parts/xor2/fu
nctional/verilog.v
ihnl/cds0/netlist
+nostdout
+nocopyright
+ncvlogargs+" -neverwarn -nostdout -nocopyright "
+ncelabargs+" -neg_tchk -nonotifier -sdf_NOCheck_celltype -access +r
-pulse_e 100 -pulse_r 100 -neverwarn -timescale 1ns/1ns -nostdout -nocopyrig
ht"
+ncsimargs+" -neverwarn -nocopyright -gui -input /home/ugrads/a/abhi.bh
attacharyya/ecen454/fulladder_run1/.simTmpNCCmd "
+mpsession+virtuoso8359
+mpshost+zach-127-01.engr.tamu.edu
-----
Relinquished control to SimVision...
ncsim>
ncsim> source /opt/coe/cadence/INCISIVE152/tools/inca/files/ncsimrc
ncsim> database -open shmWave -shm -default -into shm.db
Created default SHM database shmWave
ncsim> probe -create -shm test -all -depth 1
Created probe 1
ncsim> run
0 A=0, B=0, C=0, SUM=0, CARRY=0
50 A=1, B=0, C=0, SUM=1, CARRY=0
100 A=0, B=1, C=0, SUM=1, CARRY=0
150 A=1, B=1, C=0, SUM=0, CARRY=1
200 A=0, B=0, C=1, SUM=1, CARRY=0
250 A=1, B=0, C=1, SUM=0, CARRY=1
300 A=0, B=1, C=1, SUM=0, CARRY=1
350 A=1, B=1, C=1, SUM=1, CARRY=1
ncsim> bash-4.4$
```

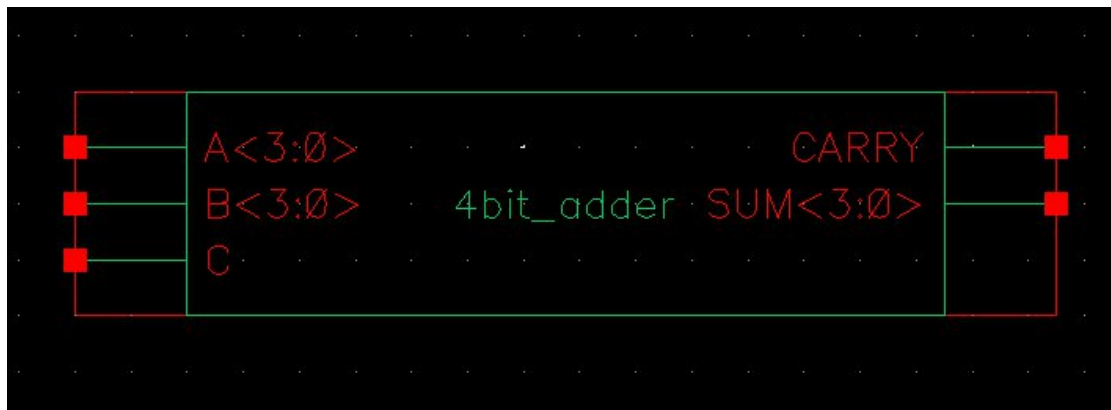
A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

All available input combinations were tested above in the simulation for the full adder. All calculations match the given truth table, so this design passes all tests in the simulation.

4-Bit Adder Schematic



4-bit Adder Symbol



4-bit Adder Test Stimulus

```
Terminal
File Edit View Search Terminal Tabs Help

/home/ugrads/a/abhi.bhattacharyya/ecen454/4bit adder run1/testfixture.verilog

// Verilog stimulus file.
// Please do not create a module in this file.

// Default verilog stimulus.

initial
begin

    A[3:0] = 4'b0000;
    B[3:0] = 4'b0000;
    C = 1'b0;

    #50
    A = 15;
    B = 15;

    #50
    A = 10;
    B = 10;
    C = 1;

    #50
    A = 5;
    B = 5;
    C = 1;

end

initial
$monitor($time," A=%b, B=%b, C=%b, SUM=%b CARRY=%b", A,B,C,SUM,CARRY);

^G Get Help ^O Write Out ^W Where Is ^K Cut Text ^J Justify ^C Cur Pos
^X Exit ^R Read File ^\ Replace ^U Uncut Text ^T To Spell ^_ Go To Line
```

4-bit Adder Sim Results

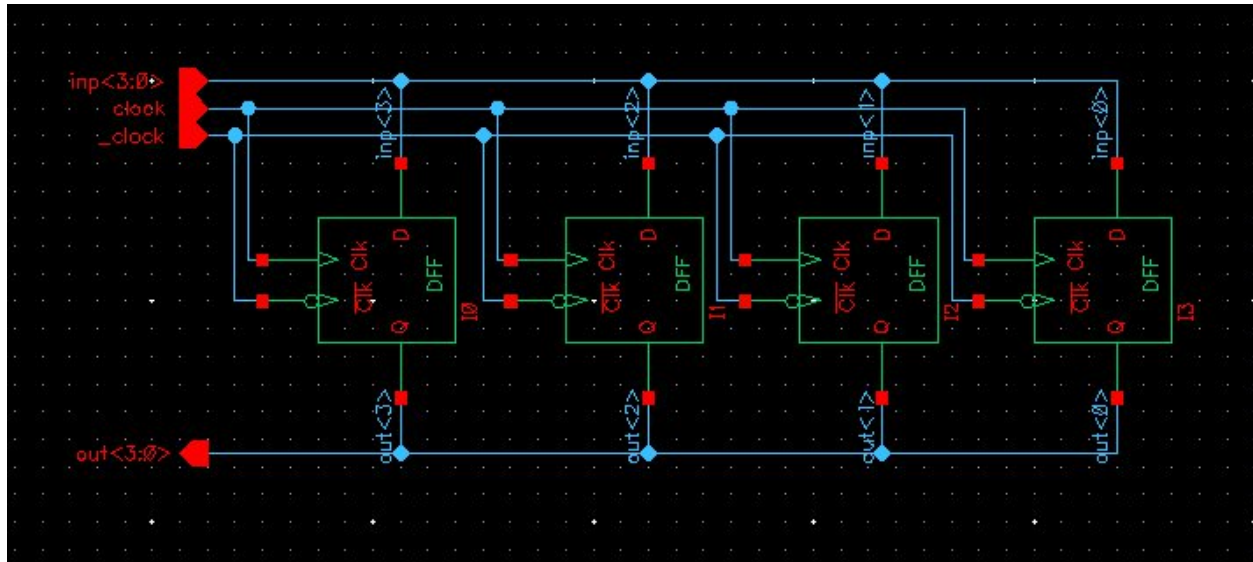
```
Terminal
File Edit View Search Terminal Tabs Help
Terminal x Terminal x Terminal x [icon] v
bash-4.4$ cat simout.tmp
TOOL: ncxlmode 15.20-s086: Started on Sep 12, 2024 at 17:00:26 CDT
ncxlmode
+delay_mode_path
+typdelays
-l
simout.tmp
/home/ugrads/a/abhi.bhattacharyya/ecen454/4bit_adder_run1/testfixture.te
mplate
-f /home/ugrads/a/abhi.bhattacharyya/ecen454/4bit_adder_run1/verilog.inp
files
/opt/coe/ncsu/ncsu-cdk-1.6.0.beta/lib/NCSU_Digital_Parts/nand2/fu
unctional/verilog.v
/opt/coe/ncsu/ncsu-cdk-1.6.0.beta/lib/NCSU_Digital_Parts/xor2/fu
nctional/verilog.v
ihnl/cds0/netlist
ihnl/cds1/netlist
+nostdout
+nocopyright
+ncvlogargs+" -neverwarn -nostdout -nocopyright "
+ncelabargs+" -neg_tchk -nonotifier -sdf_NOCheck_celltype -access +r
-pulse_e 100 -pulse_r 100 -neverwarn -timescale 1ns/1ns -nostdout -nocopyrig
ht"
+ncsimargs+" -neverwarn -nocopyright -gui -input /home/ugrads/a/abhi.bh
attacharyya/ecen454/4bit_adder_run1/.simTmpNCCmd "
+mpssession+virtuoso53972
+mpshost+zach-127-01.engr.tamu.edu
-----
Relinquished control to SimVision...
ncsim>
ncsim> source /opt/coe/cadence/INCISIVE152/tools/inca/files/ncsimrc
ncsim> database -open shmWave -shm -default -into shm.db
Created default SHM database shmWave
ncsim> probe -create -shm test -all -depth 1
Created probe 1
ncsim> run
0 A=0000, B=0000, C=0, SUM=0000 CARRY=0
50 A=1111, B=1111, C=0, SUM=1110 CARRY=1
100 A=1010, B=1010, C=1, SUM=0101 CARRY=1
150 A=0101, B=0101, C=1, SUM=1011 CARRY=0
bash-4.4$
```

The three specified cases were simulated for the 4-bit adder. The expected results were:

- $1111 + 1111 + 0 = 0000 \quad 1$
- $1010 + 1010 + 1 = 1110 \quad 1$
- $0101 + 0101 + 1 = 1011 \quad 0$

All test results from the simulation match the expected results for these given cases, so the module passes all tests.

4-Bit Register Schematic



4-Bit Register Symbol



4-Bit Register Test Stimulus



```
GNU nano 2.9.8 testfixture.verilog

// Verilog stimulus file.
// Please do not create a module in this file.

// Default verilog stimulus.

initial begin : clock_block
    clock = 0;
    forever begin
        _clock = ~clock;
        #20 clock = ~clock;
    end
end

initial begin

    inp[3:0] = 4'b0000;

    #19 inp = 5;
    #40 inp = 10;
    #2 inp = 8;
    #38 inp = 5;
    #40 inp = 3;

    #121 disable clock_block;

end

initial
$monitor($time," INPUT=%b, OUTPUT=%b, CLOCK=%b, _CLOCK=%b", inp,out,clock,_clock$

[ Wrote 32 lines ]
^G Get Help  ^O Write Out ^W Where Is  ^K Cut Text  ^J Justify   ^C Cur Pos
^X Exit      ^R Read File ^\ Replace   ^U Uncut Text ^T To Spell  ^_ Go To Line
```

4-Bit Register Sim Results

```
ncsim> source /opt/coe/cadence/INCISIVE152/tools/inca/files/ncsimrc
ncsim> database -open shmWave -shm -default -into shm.db
Created default SHM database shmWave
ncsim> probe -create -shm test -all -depth 1
Created probe 1
ncsim>
-----
Relinquished control to SimVision...
# Restoring simulation environment...
ncsim> input -quiet .reinvoke.sim
ncsim> file delete .reinvoke.sim
ncsim> run
      0 INPUT=0000, OUTPUT=xxxx, CLOCK=0, _CLOCK=1
     19 INPUT=0101, OUTPUT=xxxx, CLOCK=0, _CLOCK=1
     20 INPUT=0101, OUTPUT=0101, CLOCK=1, _CLOCK=0
     40 INPUT=0101, OUTPUT=0101, CLOCK=0, _CLOCK=1
     59 INPUT=1010, OUTPUT=0101, CLOCK=0, _CLOCK=1
     60 INPUT=1010, OUTPUT=1010, CLOCK=1, _CLOCK=0
     61 INPUT=1000, OUTPUT=1010, CLOCK=1, _CLOCK=0
     80 INPUT=1000, OUTPUT=1010, CLOCK=0, _CLOCK=1
     99 INPUT=0101, OUTPUT=1010, CLOCK=0, _CLOCK=1
    100 INPUT=0101, OUTPUT=0101, CLOCK=1, _CLOCK=0
    120 INPUT=0101, OUTPUT=0101, CLOCK=0, _CLOCK=1
    139 INPUT=0011, OUTPUT=0101, CLOCK=0, _CLOCK=1
    140 INPUT=0011, OUTPUT=0011, CLOCK=1, _CLOCK=0
    160 INPUT=0011, OUTPUT=0011, CLOCK=0, _CLOCK=1
    180 INPUT=0011, OUTPUT=0011, CLOCK=1, _CLOCK=0
    200 INPUT=0011, OUTPUT=0011, CLOCK=0, _CLOCK=1
    220 INPUT=0011, OUTPUT=0011, CLOCK=1, _CLOCK=0
    240 INPUT=0011, OUTPUT=0011, CLOCK=0, _CLOCK=1
ncsim> [abhi.bhattacharyya@olympus 4bit_register_run1]$
```

As shown in the test stimulus code, the register was simulated on a clock signal with a period of 40 time units. The first clock rising edge is at $t = 20$ time units. A race condition was observed when setting the input signal exactly on the rising edge ($20 + 40n$), where the register's output would sometimes update with the input; and other times remain unchanged. This behavior was random and nondeterministic, pointing to a race condition. To avoid this, the input was changed one time unit before the register's clock cycle.

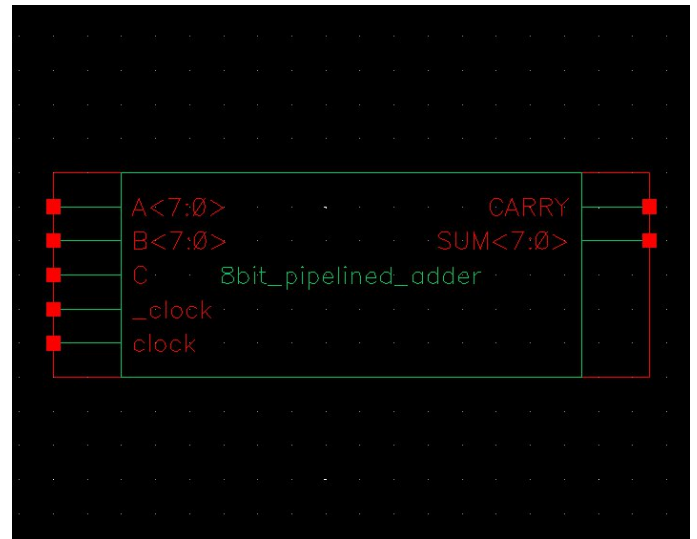
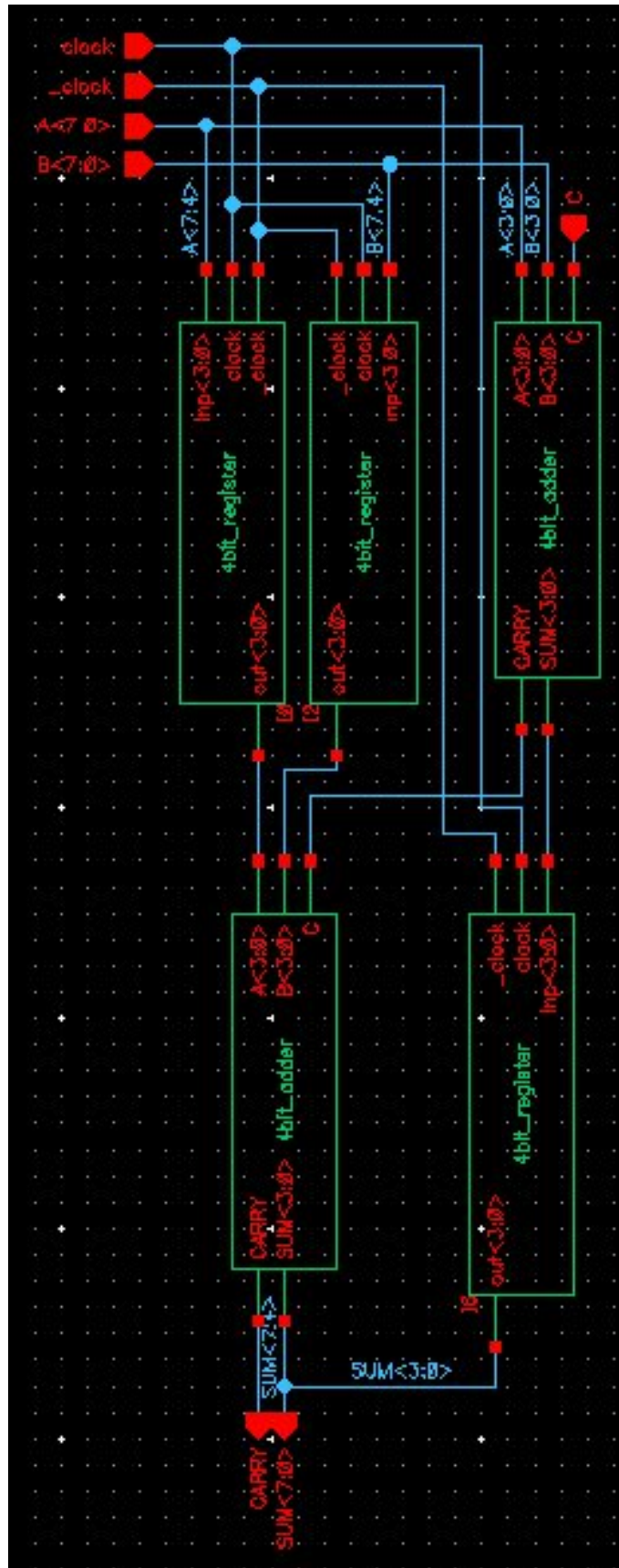
The cases tested for the 4-bit register were as follows.

Time	Input	Expected Output
------	-------	-----------------

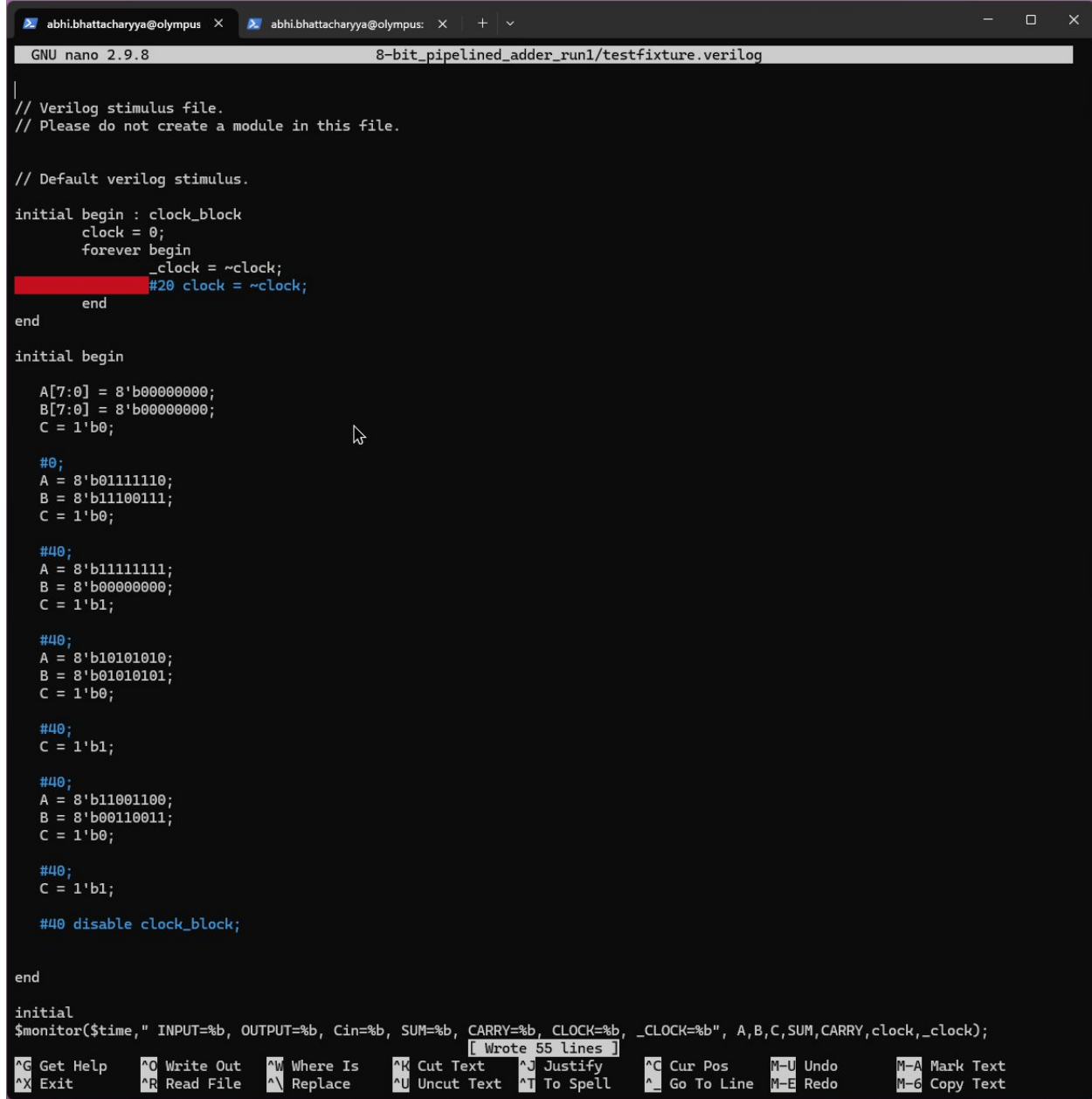
0	0000	XXXX
19	0101	
20	0101	0101
59	1010	0101
60	1010	1010
61	1000	1010 (Changing input in the middle of a clock cycle to test for issues)
99	0101	1010
100	0101	0101
139	0011	0101
140	0011	0011

The simulation results matched the expected results, so the register design worked correctly.

8-Bit Pipelined Adder Schematic & Symbol



8-Bit Pipelined Adder Test Stimulus



```
GNU nano 2.9.8 8-bit_pipelined_adder_run1/testfixture.verilog

// Verilog stimulus file.
// Please do not create a module in this file.

// Default verilog stimulus.

initial begin : clock_block
    clock = 0;
    forever begin
        _clock = ~clock;
        #20 clock = ~clock;
    end
end

initial begin

    A[7:0] = 8'b00000000;
    B[7:0] = 8'b00000000;
    C = 1'b0;

    #0;
    A = 8'b01111110;
    B = 8'b11100111;
    C = 1'b0;

    #40;
    A = 8'b11111111;
    B = 8'b00000000;
    C = 1'b1;

    #40;
    A = 8'b10101010;
    B = 8'b01010101;
    C = 1'b0;

    #40;
    C = 1'b1;

    #40;
    A = 8'b11001100;
    B = 8'b00110011;
    C = 1'b0;

    #40;
    C = 1'b1;

    #40 disable clock_block;

end

initial
$monitor($time," INPUT=%b, OUTPUT=%b, Cin=%b, SUM=%b, CARRY=%b, CLOCK=%b, _CLOCK=%b", A,B,C,SUM,CARRY,clock,_clock);

[ Wrote 55 lines ]
^G Get Help  ^O Write Out  ^W Where Is  ^K Cut Text  ^J Justify   ^C Cur Pos   M-U Undo     M-A Mark Text
^X Exit      ^R Read File  ^\ Replace   ^U Uncut Text ^T To Spell  ^_ Go To Line M-E Redo     M-G Copy Text
```

8-Bit Pipelined Adder Sim Results

```
abhi.bhattacharyya@olympus: X abhi.bhattacharyya@olympus: X + v
[abhi.bhattacharyya@olympus 8-bit_pipelined_adder_run1]$ cat simout.tmp
TOOL: ncxlmode 15.20-s086: Started on Sep 14, 2024 at 20:36:36 CDT
ncxlmode
+delay_mode_path
+typdelays
-l
simout.tmp
/home/ugrads/a/abhi.bhattacharyya/ecen454/8-bit_pipelined_adder_run1/testfixture.template
-f /home/ugrads/a/abhi.bhattacharyya/ecen454/8-bit_pipelined_adder_run1/verilog.infiles
/opt/coe/ncsu/ncsu-cdk-1.6.0.beta/lib/NCSU_Digital_Parts/nand2/functional/verilog.v
/opt/coe/ncsu/ncsu-cdk-1.6.0.beta/lib/NCSU_Digital_Parts/xor2/functional/verilog.v
/opt/coe/ncsu/ncsu-cdk-1.6.0.beta/lib/NCSU_Digital_Parts/Dlatch/behavioral/verilog.v
ihnl/cds0/netlist
ihnl/cds1/netlist
ihnl/cds2/netlist
ihnl/cds3/netlist
ihnl/cds4/netlist
+nostdout
+nocopyright
+ncvlogargs+ " -neverwarn -nostdout -nocopyright "
+ncelabargs+ " -neg_tchk -nonotifier -sdf_NOcheck_celltype -access +r -pulse_e 100 -pulse_r 100 -neverwarn -time
scale lns/lns -nostdout -nocopyright"
+ncsimargs+ " -neverwarn -nocopyright -gui -input /home/ugrads/a/abhi.bhattacharyya/ecen454/8-bit_pipelined_adder_run1
/.simTmpNCCmd "
+mpsession+virtuoso940666
+mpshost+n01-zeus.olympus.ece.tamu.edu

Relinquished control to SimVision...
ncsim>
ncsim> source /opt/coe/cadence/INCISIVE152/tools/inca/files/ncsimrc
ncsim> database -open shmWave -shm -default -into shm.db
Created default SHM database shmWave
ncsim> probe -create -shm test -all -depth 1
Created probe 1
ncsim> run
0 INPUT=01111110, OUTPUT=11100111, Cin=0, SUM=xxxxxxx, CARRY=x, CLOCK=0, _CLOCK=1
20 INPUT=01111110, OUTPUT=11100111, Cin=0, SUM=01100101, CARRY=1, CLOCK=1, _CLOCK=0
40 INPUT=11111111, OUTPUT=00000000, Cin=1, SUM=01100101, CARRY=1, CLOCK=0, _CLOCK=1
60 INPUT=11111111, OUTPUT=00000000, Cin=1, SUM=00000000, CARRY=1, CLOCK=1, _CLOCK=0
80 INPUT=10101010, OUTPUT=01010101, Cin=0, SUM=11110000, CARRY=0, CLOCK=0, _CLOCK=1
100 INPUT=10101010, OUTPUT=01010101, Cin=0, SUM=11111111, CARRY=0, CLOCK=1, _CLOCK=0
120 INPUT=10101010, OUTPUT=01010101, Cin=1, SUM=00001111, CARRY=1, CLOCK=0, _CLOCK=1
140 INPUT=10101010, OUTPUT=01010101, Cin=1, SUM=00000000, CARRY=1, CLOCK=1, _CLOCK=0
160 INPUT=11001100, OUTPUT=00110011, Cin=0, SUM=11110000, CARRY=0, CLOCK=0, _CLOCK=1
180 INPUT=11001100, OUTPUT=00110011, Cin=0, SUM=11111111, CARRY=0, CLOCK=1, _CLOCK=0
200 INPUT=11001100, OUTPUT=00110011, Cin=1, SUM=00001111, CARRY=1, CLOCK=0, _CLOCK=1
220 INPUT=11001100, OUTPUT=00110011, Cin=1, SUM=00000000, CARRY=1, CLOCK=1, _CLOCK=0
[abhi.bhattacharyya@olympus 8-bit_pipelined_adder_run1]$
```

As shown in the test stimulus code, the register was simulated on a clock signal with a period of 40 time units. The first clock rising edge is at $t = 20$ time units.

The six specified cases were simulated for the 4-bit adder. The expected results were:

Time	A	B	C	SUM	CARRY
20	1111110	11100111	0	01100101	1
60	11111111	0	1	00000000	1
100	10101010	1010101	0	11111111	0
140	10101010	1010101	1	00000000	1
180	11001100	00110011	0	11111111	0
220	11001100	00110011	1	00000000	1

Time units were added above to match the clock period (40) and initialization time ($t = 20$) used for simulation. The simulation results and calculations match the expected results. One thing of note is that the upper 4 bits of the sum update on the first clock cycle's falling edge, and the lower 4 bits update on the next clock cycle's rising edge. This is due to the layout of registers in the pipelined adder, as the design was optimized to use fewer registers. Since the output of this adder should only be read during each rising clock cycle, and the output results are correctly computed by the time of each rising clock cycle, this design satisfies the specifications for Lab 1.